# Lab 3 Tutorial: User Input and Stack Navigation

This comprehensive tutorial covers essential React Native concepts for building interactive mobile applications:

1. **Handling User Input & Forms** - Working with TextInput and form validation
2. **Introduction to Stack Navigation** - Overview of Stack Navigator
3. **Navigating Between Screens** - Implementing Stack navigation in your app
4. **Passing Data Using Navigation** - Sharing data between screens

---

## How to Run the Project

Follow these steps to set up and run the Lab 3 project:

### Step 1: Download the Repository

Download the project from GitHub:

- Repository URL: https://github.com/neonbranch/CPSC-357-Lab

### Step 2: Extract and Navigate

1. Extract the downloaded files
2. Navigate to the `lab3` folder
3. Then navigate to the `expo-app-v2` folder

```
cd lab3
cd expo-app-v2
```

### Step 3: Install Dependencies

Install all required npm packages:

```
npm install
```

### Step 4: Start the Expo Development Server

Run the Expo development server:

```
npx expo start
```

---

# 1. Handling User Input & Forms

## Login Form Example

Here's a complete login form example that demonstrates handling user input with validation:

```
import { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  Button,
  StyleSheet,
  Alert,
} from 'react-native';
import { useNavigation } from '@react-navigation/native';

export default function LoginForm() {
  const navigation = useNavigation();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = () => {
    // Validation: Check if fields are empty
    if (!email || !password) {
      Alert.alert('Error', 'Please fill in all fields');
      return;
    }

    // Length validation: min 5, max 20 characters
    if (email.length < 5 || email.length > 20) {
      Alert.alert('Error', 'Email must be between 5 and 20 characters');
      return;
    }
    if (password.length < 5 || password.length > 20) {
      Alert.alert('Error', 'Password must be between 5 and 20 characters');
      return;
    }

    // Assume: Login is successful
    Alert.alert('Success', `Welcome, ${email}!`);

    // Navigate to Home with email and loginStatus
    navigation.navigate('Home', {
      email: email,
      loginStatus: true
    });

    // Reset form
    setEmail('');
    setPassword('');
  };
```

```
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Login</Text>

      <Text style={styles.label}>Email</Text>
      <TextInput
        style={styles.input}
        placeholder="Enter your email"
        keyboardType="email-address"
        autoCapitalize="none"
        value={email}
        onChangeText={setEmail}
      />

      <Text style={styles.label}>Password</Text>
      <TextInput
        style={styles.input}
        placeholder="Enter your password"
        secureTextEntry={true}
        value={password}
        onChangeText={setPassword}
      />

      <Button title="Login" onPress={handleSubmit} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#fff',
  },
  title: {
    fontSize: 28,
    fontWeight: 'bold',
    marginBottom: 30,
    textAlign: 'center',
  },
  label: {
    fontSize: 16,
    fontWeight: '600',
    marginBottom: 8,
    marginTop: 15,
  },
  input: {
    height: 50,
    borderWidth: 1,
    borderColor: '#ddd',
    borderRadius: 8,
    paddingHorizontal: 15,
    fontSize: 16,
```

```
    backgroundColor: '#f9f9f9',
  },
});
```

**Key Features of this Login Form:**

- Uses `useState` to manage email and password state
- Uses `useNavigation` hook to navigate after successful login
- Includes form validation (empty fields and length checks)
- Uses `secureTextEntry` for password field
- Uses `keyboardType="email-address"` for email input
- Resets form fields after submission
- Passes data to the next screen via navigation params

---

## 2. Introduction to Stack Navigation

Navigation allows users to move between different screens in your app. React Navigation is the most popular navigation library for React Native.

### Stack Navigator

The Stack Navigator navigates between screens like a stack (last-in-first-out). It's common for hierarchical navigation.

**Characteristics:**

- Push new screens on top
- Pop screens to go back
- Shows header with back button
- Good for: Detail screens, forms, authentication flows

### Navigation Container

All navigators must be wrapped in a `NavigationContainer`:

```
import { NavigationContainer } from '@react-navigation/native';

export default function App() {
  return (
    <NavigationContainer>
      {/* Your navigator here */}
    </NavigationContainer>
  );
}
```

---

## 3. Navigating Between Screens

## Simple Stack Navigation Example with Passing Data

Here's a complete example showing Stack Navigation with data passing:

```
import 'react-native-gesture-handler';
import React from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { useNavigation, useRoute } from '@react-navigation/native';

const Stack = createNativeStackNavigator();

// Screen 1: Home Screen - Passes data to Details
function HomeScreen() {
  const navigation = useNavigation();

  const goToDetails = () => {
    // Navigate and pass data
    navigation.navigate('Details', {
      userName: 'John Doe',
      userId: 123
    });
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Home Screen</Text>
      <Button title="Go to Details" onPress={goToDetails} />
    </View>
  );
}

// Screen 2: Details Screen - Receives data
function DetailsScreen() {
  const route = useRoute();
  const navigation = useNavigation();

  // Get data from route params
  const userName = route.params?.userName || 'Guest';
  const userId = route.params?.userId || 0;

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Details Screen</Text>
      <Text>User Name: {userName}</Text>
      <Text>User ID: {userId}</Text>
      <Button title="Go Back" onPress={() => navigation.goBack()} />
    </View>
  );
}
```

```
// Main App Component
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
  },
});
```

**Key Points:**

- `useNavigation()` hook is used to navigate between screens
- `useRoute()` hook is used to receive data from previous screen
- Data is passed using the second parameter of `navigation.navigate()`
- Use optional chaining (`?.`) to safely access route params

---

# 4. Passing Data Using Navigation

## How to Pass Data

**Step 1: Pass data when navigating**

```
import { useNavigation } from '@react-navigation/native';

function HomeScreen() {
  const navigation = useNavigation();

  const handlePress = () => {
    // Pass data as second parameter
    navigation.navigate('Details', {
      userName: 'John Doe',
      userId: 123
    });
```

```
  };

  return (
    <Button title="Go to Details" onPress={handlePress} />
  );
}
```

**Step 2: Receive data in the destination screen**

```
import { useRoute } from '@react-navigation/native';

function DetailsScreen() {
  const route = useRoute();

  // Get data using optional chaining
  const userName = route.params?.userName || 'Guest';
  const userId = route.params?.userId || 0;

  return (
    <View>
      <Text>Name: {userName}</Text>
      <Text>ID: {userId}</Text>
    </View>
  );
}
```

### Navigation Methods

```
// Navigate to a screen
navigation.navigate('ScreenName');

// Navigate with data
navigation.navigate('Details', { userName: 'John', userId: 123 });

// Go back
navigation.goBack();
```

---

## Summary

This tutorial covered essential concepts for building interactive React Native applications:

- **User Input & Forms**: Handling text input and form validation
- **Stack Navigation**: Understanding and implementing Stack Navigator
- **Screen Navigation**: Moving between different screens using Stack Navigator
- **Data Passing**: Sharing data between screens using navigation params

## Next Steps

- Practice building forms with validation
- Create a multi-screen app with Stack navigation
- Explore advanced Stack navigation patterns
- Study React Navigation documentation: https://reactnavigation.org

## Additional Resources

- React Navigation Documentation
- React Native TextInput